

# **“Smart Multi-Vendor E-Commerce Marketplace”**

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

**By**

**G. NEHA DEEPIKA : 2520030068**

**M. THARA VARSHINI : 2520030052**

Under the Esteemed Guidance of

**MR. SATEESH KUMAR**

**Course Instructor**

**Database Systems Engineering and Distributed Backend Development**



## **Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University  
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



**Declaration**

The Project Report entitled “SMART MULTI-VENDOR E-COMMERCE MARKETPLACE” is a record of bonafide work of G. NEHA DEEPIKA - 2520030068 and M. THARA VARSHINI - 2520030052 submitted in partial fulfillment for the award of B.Tech in Computer Science Engineering / Computer Science and Information Technology at K L (Deemed to be) University. The work presented in this report has been prepared for the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E) and represents the project work carried out by the above team.

**G. NEHA DEEPIKA – 2520030068**

**M. THARA VARSHINI – 2520030052**

**K L (Deemed to be) University**  
**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



**CERTIFICATE**

This is to certify that the mini project based report entitled “SMART MULTI-VENDOR E-COMMERCE MARKETPLACE” is a bonafide work done and submitted by G. NEHA DEEPIKA - 2520030068 and M. THARA VARSHINI - 2520030052 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Computer Science Engineering / Computer Science and Information Technology, during the academic year 2025–2026 | TRIMESTER 4. The project was carried out under the academic guidance of MR. SATEESH KUMAR for the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E).

**Signature of the Course Instructor**

**Signature of the Course Coordinator**

**Signature of the HOD**

## **ACKNOWLEDGEMENT**

The successful completion of this project would not have been possible without the guidance, encouragement, and support of many people. We express our sincere gratitude to everyone who contributed directly or indirectly to the completion of this project report.

We place on record our deepest appreciation to our Course Instructor MR. SATEESH KUMAR for the valuable guidance, technical direction, continuous feedback, and encouragement provided throughout the development of the Smart Multi-Vendor E-Commerce Marketplace. The discussions on database design, distributed backend processing, transaction handling, and API testing helped us understand how academic concepts can be translated into a practical software system.

We are grateful to the Department of Computer Science Engineering and the Department of Computer Science and Information Technology for providing the academic environment and facilities required for carrying out the project. We also thank the teaching and non-teaching staff members for their support during development, documentation, and evaluation.

Finally, we thank our parents, friends, classmates, and all those who supported us with suggestions, testing assistance, and encouragement. Their cooperation helped us complete the project with greater confidence and discipline.

**G. NEHA DEEPIKA – 2520030068**

**M. THARA VARSHINI – 2520030052**

## **Abstract**

The Smart Multi-Vendor E-Commerce Marketplace is a database-driven web application designed to provide a unified shopping environment in which customers can discover products from multiple vendors, compare offers, evaluate vendor reliability, and complete multi-vendor purchases through a single interface. Unlike conventional e-commerce systems that primarily focus on product listing and checkout, the proposed marketplace combines transactional data management with analytics-oriented features such as Smart Multi-Vendor Matching, Vendor Trust Score, Multi-Vendor Cart, Price History Analysis, Product Recommendations, and Smart Inventory Rules. The system uses a hybrid database approach. MySQL is used for structured and transaction-critical data including users, vendors, products, inventory, orders, payments, and order details, while MongoDB is used for flexible or semi-structured information such as shopping activity, search history, product views, recommendation records, notifications, and configuration-oriented data. The Smart Vendor Matching module evaluates vendors using factors such as price, availability, rating, delivery performance, return rate, and order completion. The Vendor Trust Score provides a consolidated reliability indicator derived from operational and customer-facing metrics. The Multi-Vendor Cart allows a customer to select products from different vendors in one shopping session and automatically organizes the final order into vendor-specific sub-orders. Price History Analysis records historical price information to support fair-price insights, while sales-pattern analysis is used to generate inventory alerts. The platform is developed using HTML, CSS, JavaScript, and Bootstrap on the frontend and Node.js with Express.js on the backend. MySQL Workbench and MongoDB Compass are used as database tools, Postman is used for API testing, Visual Studio Code is used as the development environment, and Git with GitHub supports version control and collaboration. Overall, the project demonstrates how relational and NoSQL databases, backend APIs, data analytics, and marketplace logic can be combined to build a scalable and intelligent multi-vendor commerce platform.

## Index

| S. No. | Chapters / Topics                                                                         | Page  |
|--------|-------------------------------------------------------------------------------------------|-------|
|        | Acknowledgement                                                                           | 4     |
|        | Abstract                                                                                  | 5     |
| 1      | Introduction — Background, Problem Statement, Scope, Objectives, Importance, Target Users | 7–10  |
| 2      | System Requirements — Hardware, Software, Development Tools                               | 11–12 |
| 3      | Technology Stack — Front-End, Back-End, Database, Version Control, APIs                   | 13–14 |
| 4      | System Architecture — High-Level Architecture, Layers, Deployment                         | 15–16 |
| 5      | Design — Data Flow, ER Diagram / Database Schema, UI and Marketplace Logic                | 17–19 |
| 6      | Implementation — Modules, Front-End Logic, API Integration, Authentication                | 20–21 |
| 7      | Features — Main Features and Technical/User Perspective                                   | 22–23 |
| 8      | Testing — Postman, Integration Testing, Test Tools, Test Cases                            | 24–25 |
| 9      | Deployment — Steps, Environment Configuration, Hosting                                    | 26    |
| 10     | Challenges & Limitations — Issues, Solutions, Current Limitations                         | 27–28 |
| 11     | Future Enhancements — Planned Features, Integrations, Optimizations                       | 29    |
| 12     | Conclusion — Summary, Achievements, Skills Learned                                        | 30–31 |
| 13     | References                                                                                | 32    |
| 14     | Appendices — Code, Setup, User Guide, Review Form                                         | 33–36 |

# CHAPTER - 1 INTRODUCTION

## 1.1 Background of the Project

Electronic commerce has evolved from simple online catalogues into complex marketplaces in which customers, independent sellers, logistics providers, payment services, and data platforms interact continuously. A multi-vendor marketplace is particularly valuable because it allows a customer to compare products and offers from several sellers without visiting separate stores. However, a marketplace that simply aggregates products does not automatically provide intelligent vendor selection, transaction consistency, or useful price and inventory insights.

The proposed Smart Multi-Vendor E-Commerce Marketplace is designed around the idea that product discovery, vendor comparison, purchasing, and post-purchase information should operate as one coordinated system. The platform maintains structured transactional records in MySQL and uses MongoDB for flexible behavioural and catalogue-oriented information. This division allows strongly consistent operations such as orders and payments to be handled using relational constraints while activity-oriented records can evolve without frequent schema changes.

A central feature of the project is Smart Multi-Vendor Matching. When the same or similar product is available from multiple sellers, the system can evaluate vendor attributes including offered price, current availability, rating, delivery performance, return rate, and order completion. This transforms a basic product search into a marketplace decision-support process. The Vendor Trust Score complements this feature by presenting a consolidated view of vendor reliability based on measurable operational indicators.

The platform also addresses the complexity of purchasing from multiple sellers. A Multi-Vendor Cart permits products from different vendors to coexist in a single customer cart. At checkout, the system separates the purchase into vendor-specific order groups while retaining one overall checkout experience. Price History Analysis and Smart Inventory Rules further extend the system from a transactional application into an analytics-oriented marketplace.

The project therefore brings together database management, distributed backend processing, REST APIs, web interface development, data analytics, authentication, and version-controlled software engineering. The design follows the project requirements specified for Database Systems Engineering and Distributed Backend Development.

## 1.2 Problem Statement

Traditional single-vendor shopping systems provide limited support when a product is sold by several independent vendors. Customers may need to compare prices manually, inspect seller ratings separately, and decide whether a lower price is worth the potential difference in delivery reliability or return experience. The absence of a consolidated vendor comparison mechanism can increase search time and make purchase decisions less transparent.

A second challenge is transaction management for multi-vendor purchases. If a customer selects products from several vendors, the system must maintain a single cart experience while preserving vendor-specific order details, inventory changes, payment allocation, and order status. Poorly designed relationships between cart, order, vendor, and product entities can lead to inconsistent records or incomplete transactions.

A third problem is the lack of historical context. A product's current price does not tell a customer whether the price is relatively high or low compared with its earlier values. Similarly, vendors may not receive useful early warnings when stock movement indicates a potential shortage. A database-driven system can convert these historical and behavioural records into practical insights.

The project addresses these issues by combining relational transaction processing, flexible activity storage, vendor scoring, recommendation logic, price history analysis, and inventory rules in one web-based marketplace.

- Provide a unified marketplace for multiple independent vendors.
- Enable intelligent comparison of vendor offers for the same product.
- Support a single cart and checkout flow for products from multiple vendors.
- Maintain reliable transaction records for orders, payments, and inventory.
- Generate fair-price insights using historical price information.
- Use sales patterns to create actionable inventory alerts.

### **1.3 Scope of the Project**

The scope covers the design and development of a responsive web application for customers and marketplace administrators, with vendor-oriented data structures and processing logic. The customer side includes product search, vendor comparison, cart management, checkout, recommendations, price-history viewing, and order tracking. The backend manages authentication, product and vendor information, inventory, order creation, payment records, and analytics-oriented operations.

The database scope includes structured entities in MySQL and flexible collections in MongoDB. MySQL is responsible for transactional entities where relationships and consistency are central. MongoDB supports activity-oriented information such as search history, product views, recommendation events, notifications, and flexible catalogue metadata. The two stores communicate through the Node.js backend rather than exposing database access directly to the client.

The project scope also includes API development and testing using Express.js and Postman. Development is performed using Visual Studio Code, with Git and GitHub used for source-code management. The initial system is intended as an academic prototype that can be expanded to support production payment gateways, external logistics APIs, advanced recommendation models, and cloud deployment.

### **1.4 Objectives**

- Design a multi-vendor e-commerce marketplace with a clear and responsive user interface.
- Use MySQL for transaction-critical records and MongoDB for flexible behavioural or semi-structured data.
- Implement Smart Multi-Vendor Matching using price, availability, ratings, delivery performance, return rate, and order completion.
- Calculate a Vendor Trust Score from measurable seller-performance indicators.
- Allow customers to create a Multi-Vendor Cart and complete a unified checkout.
- Maintain product price history and expose historical price insights.
- Generate Smart Inventory Rules and alerts from sales patterns and stock levels.
- Provide secure authentication and role-aware backend operations.
- Build and validate REST APIs using Node.js, Express.js, and Postman.
- Demonstrate practical application of database systems and distributed backend concepts.

### **1.5 Importance of the Application**

The application is important because it treats an e-commerce marketplace as a data-management problem rather than only a product-listing problem. A marketplace contains many-to-many relationships between products and vendors, repeated transactions, changing inventory, user behaviour, and time-dependent prices. Managing these elements systematically can improve consistency and make the stored data useful for decision support.

For customers, the marketplace reduces repetitive comparison work by bringing multiple vendor offers together. The Trust Score provides additional context beyond a single star rating, while price history gives temporal context to current prices. The Multi-Vendor Cart simplifies purchases that would otherwise require separate transactions.

For vendors and administrators, structured order and inventory records make it possible to monitor sales patterns and identify stock risks. The architecture also provides a foundation for later analytics, vendor dashboards, and recommendation models. From an academic perspective, the project demonstrates the combined use of relational and NoSQL databases, API design, backend modularity, and data-driven application features.

### **1.6 Target Users / Audience**

- Customers who want to compare offers from several vendors.
- Independent vendors who sell products through a shared marketplace.
- Marketplace administrators who manage products, vendors, orders, and system rules.
- Students and developers studying hybrid database architectures and backend systems.

- Organizations interested in extending the prototype into a larger marketplace platform.

## CHAPTER - 2 SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

Development requires a computer capable of running Visual Studio Code, Node.js, a web browser, MySQL services, MongoDB services, and database management tools concurrently. A modern multi-core processor and at least 8 GB RAM are suitable for the academic prototype; 16 GB RAM provides additional comfort when both databases and development utilities are active.

Storage should provide sufficient space for the source repository, Node.js dependencies, database files, logs, and project documentation. An SSD is preferred because database access and package installation involve frequent read/write operations. A stable internet connection is required for downloading dependencies, using GitHub, testing APIs, and accessing any external services connected to the project.

End users do not require specialized hardware. The application is browser-based and can be accessed using a desktop, laptop, tablet, or modern smartphone with a current web browser and stable internet connection.

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent.
- Memory: 8 GB minimum; 16 GB recommended for development.
- Storage: 250 GB or more, preferably SSD.
- Display: 1366×768 minimum; Full HD recommended.
- Network: stable broadband connection.
- Client: modern Chrome, Edge, Firefox, or equivalent browser.

### 2.2 Software Requirements

The frontend is based on standard web technologies: HTML5 for structure, CSS3 for presentation, JavaScript for client-side logic, and Bootstrap for responsive components. The backend uses Node.js as the runtime and Express.js for REST API development. The database layer uses MySQL and MongoDB according to the data characteristics identified in the project design.

MySQL Workbench is used to create and inspect relational schemas, execute SQL queries, and review relationships. MongoDB Compass provides a graphical interface for inspecting collections and documents. Postman is used to send API requests, verify responses, and test error conditions. Visual Studio Code serves as the main development environment. Git and GitHub provide version tracking and collaboration.

The application can be developed on Windows 10/11, while the same Node.js-based backend can later be deployed to a Linux-based server or cloud platform.

### 2.3 Development Tools and Frameworks

- Frontend: HTML5, CSS3, JavaScript, Bootstrap.

- Backend: Node.js and Express.js.
- Relational database: MySQL.
- NoSQL database: MongoDB.
- Database tools: MySQL Workbench and MongoDB Compass.
- API testing: Postman.
- IDE: Visual Studio Code.
- Version control: Git and GitHub.
- Runtime environment: Node.js.
- Operating system: Windows 10/11.

## CHAPTER - 3 TECHNOLOGY STACK

### 3.1 Front-End Design

The frontend provides the customer-facing layer of the marketplace. HTML5 defines the page structure for navigation, product sections, vendor information, cart content, checkout forms, and account views. CSS3 controls layout, spacing, typography, responsiveness, and visual feedback. Bootstrap provides reusable responsive components so that the interface can adapt to desktop and mobile screen sizes.

JavaScript manages dynamic behaviour such as product filtering, cart updates, vendor comparisons, validation, asynchronous API requests, and display of recommendation results. The frontend communicates with Express.js endpoints through HTTP requests and consumes JSON responses. Client-side validation reduces invalid requests, while server-side validation remains the authoritative security boundary.

The interface is organized around common e-commerce workflows: browse, search, compare, add to cart, review cart, checkout, and view orders. Marketplace-specific components include vendor comparison cards, Trust Score indicators, price-history panels, and inventory-related status information.

### 3.2 Back-End Design

Node.js provides the server runtime and Express.js organizes REST endpoints, middleware, validation, authentication, and business logic. The backend is responsible for enforcing marketplace rules rather than allowing the browser to directly manipulate database records. This separation improves maintainability and protects transactional operations.

The backend can be divided into logical modules such as authentication, users, vendors, products, inventory, carts, orders, payments, recommendations, price history, and notifications. Each module exposes focused functions and routes. Shared middleware can handle authentication, request validation, error handling, logging, and response formatting.

The backend coordinates both database systems. A request that requires transactional information is handled through MySQL operations, while activity-oriented records can be written to MongoDB. The API layer therefore acts as the integration boundary between the browser and the hybrid data store.

### 3.3 Database Design

The database strategy is hybrid. MySQL stores structured records whose relationships and transactional consistency are important: users, vendors, products, inventory, orders, order details, payments, and other core commerce entities. Foreign keys and constraints can be used to preserve referential integrity.

MongoDB stores flexible information that may change structure more frequently: shopping activity, search history, product views, recommendation records, notifications, and configuration data. Documents allow new attributes to be introduced without changing a relational schema for every behavioural event.

The two databases are not treated as competing alternatives. They serve different data characteristics. The backend determines which store is appropriate for each operation and can combine results before returning a unified response to the frontend.

### **3.4 Version Control**

Git is used to track source-code changes throughout development. The project can be maintained in a GitHub repository with separate branches for stable code and feature development. Commits should describe meaningful changes, while pull requests can be used for review before merging shared work.

Version control is particularly useful for a two-member team because it provides a recoverable history and makes it easier to identify which change introduced a defect. Environment secrets such as database passwords and API keys should never be committed to the repository.

### **3.5 APIs and External Services**

The project uses REST APIs as the communication mechanism between the frontend and backend. Postman is used to test endpoints independently of the user interface. The design can later be extended to external payment, logistics, product-catalogue, or notification services without changing the browser-to-backend contract.

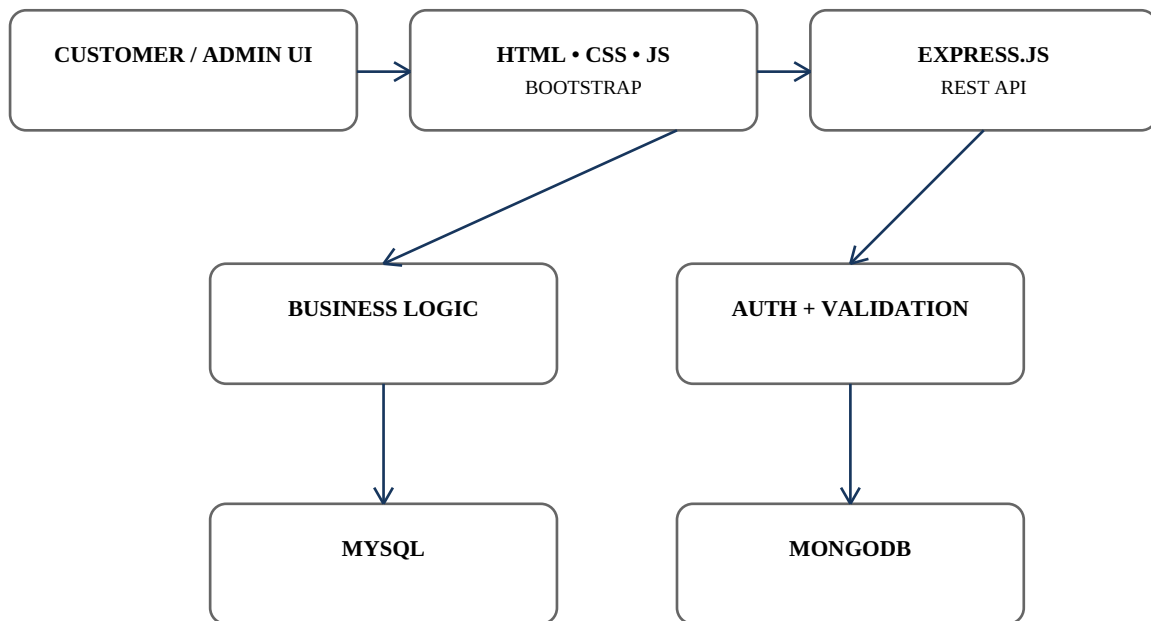
API responses are exchanged in JSON format. Standard HTTP methods are used according to operation type: GET for retrieval, POST for creation, PUT/PATCH for updates, and DELETE where removal is permitted. Authentication tokens or sessions can be used to protect customer and administrative routes.

## CHAPTER - 4 SYSTEM ARCHITECTURE

### 4.1 High-Level Architecture Diagram

The architecture follows a layered web application model. Users interact with the browser-based interface, which communicates with Express.js REST APIs. The backend contains business rules, validation, authentication, marketplace algorithms, and database access. MySQL and MongoDB form the persistence layer. This arrangement separates presentation concerns from transaction processing and data storage.

*High-Level Architecture of the Smart Marketplace*



*Figure 4.1: High-level architecture of the proposed marketplace*

### 4.2 Description of Each Layer

**Presentation layer:** HTML, CSS, JavaScript, and Bootstrap provide the user interface. Product listings, vendor comparisons, cart contents, checkout forms, and account information are rendered here.

**API and application layer:** Node.js and Express.js expose REST endpoints. This layer validates input, checks authorization, executes marketplace logic, coordinates database operations, and returns JSON responses.

**Data layer:** MySQL stores structured transactional records. MongoDB stores flexible activity and recommendation information. Database access is encapsulated in backend modules so that clients never connect directly to the databases.

Analytics and rules layer: Vendor matching, Trust Score calculation, price-history analysis, product recommendations, and inventory rules operate on stored data and return decision-support information to the application.

### **4.3 Deployment Architecture**

During development, the frontend, Express server, MySQL service, and MongoDB service can run locally on the development machine. Postman is used to test backend endpoints independently. GitHub stores the source repository and supports collaboration.

For a production-style deployment, the frontend can be hosted on a static web host while the Node.js backend runs on a cloud application server. MySQL can be hosted as a managed relational database and MongoDB as a managed document database. Environment variables are used for connection strings, credentials, and secrets. A reverse proxy or managed gateway can terminate HTTPS and route requests to the backend.

A CI/CD pipeline can later automate dependency installation, testing, build validation, and deployment after changes are merged into the main branch. Backups and monitoring should be configured for both database systems.

## CHAPTER - 5 DESIGN

### 5.1 Data Flow and Marketplace Workflow

A typical customer workflow begins with registration or login, followed by product search or category browsing. The frontend requests product and vendor information from the backend. The backend retrieves structured product, vendor, inventory, and price information and can enrich the response with recommendation or activity data from MongoDB.

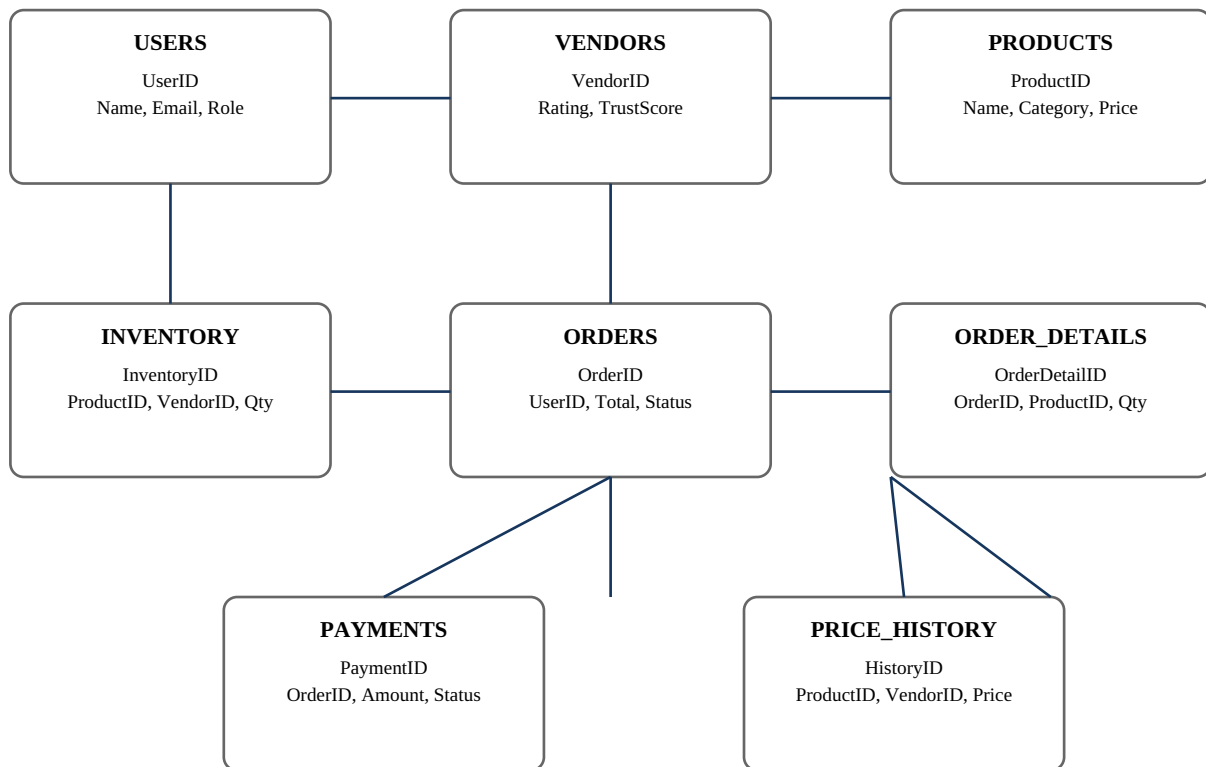
When multiple vendors sell a comparable product, the Smart Multi-Vendor Matching function evaluates the available offers. The result is presented with price, availability, vendor rating, Trust Score, delivery performance, return rate, and order-completion indicators. The customer can then add one or more offers to the cart.

The Multi-Vendor Cart retains vendor identity for each cart line. During checkout, the backend validates stock and price, creates an overall order record, and generates vendor-specific order details or sub-orders. Payment status is recorded and inventory quantities are updated as part of the transaction workflow.

### 5.2 ER Diagram / Database Schema

The conceptual schema separates users, vendors, products, inventory, orders, order details, payments, and price history. A user can create many orders. A vendor can offer many products, and a product can be associated with multiple vendors through inventory or offer records. An order contains multiple order-detail records, each identifying the product, vendor, quantity, and applicable price.

The relational model is designed to keep transactional data consistent. Foreign keys connect dependent records, while indexes can be placed on product names, vendor identifiers, order dates, user email addresses, and frequently searched fields. Price-history records are time-stamped so that historical trends can be calculated.



*Conceptual relational schema; final implementation may extend these entities.*

*Figure 5.1: Conceptual ER diagram / database schema*

### 5.3 Smart Vendor Matching Design

The matching logic converts multiple vendor attributes into a comparable result. A practical weighted model can normalize each factor to a common scale. Price can be transformed so that lower values receive higher scores, while availability, rating, delivery performance, return behaviour, and order completion can be normalized directly.

A sample conceptual score is:  $\text{VendorMatchScore} = w_1 \times \text{PriceScore} + w_2 \times \text{AvailabilityScore} + w_3 \times \text{RatingScore} + w_4 \times \text{DeliveryScore} + w_5 \times \text{ReturnScore} + w_6 \times \text{CompletionScore}$ . The weights are configurable and should be tuned using project requirements or later user feedback. The algorithm is a decision-support mechanism; it does not permanently change vendor records.

### 5.4 Vendor Trust Score Design

The Vendor Trust Score summarizes seller reliability. It can combine verified order-completion rate, average customer rating, delivery performance, return behaviour, and the volume of successfully completed transactions. The score should be calculated from measurable records and should be recalculated as new transactions are recorded.

A Trust Score is different from a single product rating. Product ratings reflect customer feedback for purchased items, while the Trust Score is intended to summarize vendor-level operational reliability. The

interface should show the component indicators or an explanation so that the score remains understandable.

## **5.5 UI/UX Design**

The interface follows a marketplace-oriented layout with a clear navigation bar, search controls, product cards, vendor comparison sections, cart access, and account pages. Important information such as price, stock availability, vendor name, rating, and Trust Score should be visible without requiring excessive navigation.

Bootstrap responsive components allow the same interface to adapt to different screen sizes. Forms use validation messages and clear labels. Checkout pages should present vendor-specific groups so customers understand that a single cart may contain items fulfilled by different sellers.

## CHAPTER - 6 IMPLEMENTATION

### 6.1 Module-Wise Implementation

User and Authentication Module: manages registration, login, profile information, and role-aware access. Passwords are stored using secure hashing rather than plain text. Protected routes require authenticated requests.

Vendor Module: maintains vendor profiles, ratings, performance indicators, return statistics, and offer availability. Vendor information is used by matching and Trust Score calculations.

Product Module: stores product identity, category, descriptions, and product-level attributes. Multiple vendors can be associated with the same product.

Inventory Module: maintains stock quantities and supports rule-based alerts when sales patterns indicate that available inventory may fall below a configured threshold.

Cart and Order Module: maintains customer cart lines, preserves vendor identity, validates stock at checkout, and generates order/sub-order structures.

Price History Module: records time-stamped product/vendor prices and supports historical comparisons.

Recommendation Module: uses product views, search history, purchase patterns, and related-product relationships to produce recommendations.

Notification Module: stores or publishes messages related to order status, inventory alerts, and recommendation events.

### 6.2 Front-End Logic and State Management

The frontend maintains temporary application state such as search filters, selected vendor offer, cart contents, and authentication status. JavaScript event handlers respond to user actions and issue asynchronous API requests. The response is converted into UI updates without requiring a complete page refresh.

Client-side state must not be treated as authoritative for price, inventory, or payment information. At checkout, the server rechecks the relevant database records because prices or stock may have changed after the user added an item to the cart.

### 6.3 API Integration

Express.js routes expose operations such as user registration, login, product search, vendor comparison, cart creation, checkout, order retrieval, price-history retrieval, recommendation retrieval, and inventory alerts. The backend returns consistent JSON structures containing status information and data or error messages.

Postman collections can be organized by module. Each request should be tested with valid inputs, invalid inputs, missing authentication, nonexistent identifiers, and boundary values. This helps verify that the API behaves correctly before frontend integration.

#### **6.4 Authentication & Authorization**

Authentication verifies who the user is, while authorization determines which operations that user can perform. Customer routes can allow profile, cart, checkout, and order operations, whereas administrative routes can be restricted to authorized roles. Vendor operations should be scoped so a vendor cannot modify another vendor's records.

The backend should validate authentication credentials and issue a secure session or token. Sensitive values such as database credentials and token secrets are kept in environment variables. Input validation is applied before database operations, and error responses avoid exposing internal implementation details.

#### **6.5 Multi-Vendor Checkout Processing**

At checkout, the server reads the cart and groups lines by vendor. It verifies that every item still exists, the requested quantity is available, and the current price is acceptable. The overall purchase can then be represented as a parent order with vendor-specific child records, or as separate vendor orders linked to one checkout reference.

This design allows each vendor to manage its own fulfilment status while the customer still sees one consolidated purchase. Payment and order status should be updated only after the backend confirms the relevant transaction result. Database transactions are used where operations must succeed or roll back together.

## CHAPTER - 7 FEATURES

### 7.1 List of Main Features

- Multi-vendor product catalogue and search.
- Smart Multi-Vendor Matching.
- Vendor Trust Score.
- Multi-Vendor Cart and unified checkout.
- Price History Analysis.
- Product Recommendations.
- Smart Inventory Rules and alerts.
- User authentication and profile management.
- Order and payment record management.
- Responsive Bootstrap-based web interface.
- REST APIs for frontend/backend communication.
- Hybrid MySQL + MongoDB data management.

### 7.2 How Each Feature Works — User Perspective and Technical Description

**Multi-Vendor Matching:** The customer selects a product and sees available vendor offers together. The backend normalizes vendor metrics and calculates a comparison score. The UI can show the most relevant factors next to each offer.

**Vendor Trust Score:** The customer can inspect a seller's Trust Score along with rating and operational indicators. The backend derives the score from stored performance records.

**Multi-Vendor Cart:** Products from different vendors can be added to the same cart. Cart lines retain vendor identity. At checkout, the backend groups the lines into vendor-specific order records.

**Price History Analysis:** Historical price points are stored with timestamps. The application can display recent values, minimum/maximum observations, and a simple trend indicator. This helps contextualize the current offer.

**Product Recommendations:** The system can use search history, product views, related products, and past purchases to produce suggestions. MongoDB is suitable for storing these flexible behavioural events.

**Smart Inventory Rules:** The system monitors stock quantity and sales movement. When configured conditions are met, it creates an alert for replenishment or review.

Authentication: Login and protected routes ensure that personal order information is not exposed to unauthenticated users.

Order Management: Customers can view order details and status, while backend records maintain vendor-specific fulfillment information.

### **7.3 Feature Interaction**

The features are designed to reinforce one another. A customer search can create a product-view or search-history event, which can later support recommendations. Vendor comparison uses inventory and performance records. A selected offer enters the multi-vendor cart, which creates order and payment records. Completed orders update sales information, which can influence future Trust Scores and inventory alerts. Price records continue to provide historical context for later searches.

## CHAPTER - 8 TESTING

### 8.1 Postman Testing — Frontend/Backend

Postman is used to test the REST API independently of the browser. This is important because backend defects can otherwise be confused with frontend rendering problems. Requests are organized by module and executed using representative customer, vendor, and administrator data.

Authentication endpoints are tested first so that protected requests can be supplied with valid credentials. Product and vendor endpoints are then checked for filtering, missing parameters, invalid identifiers, and empty-result conditions. Cart and checkout endpoints receive special attention because they modify transactional data.

### 8.2 Integration Testing

Integration testing verifies that modules work together rather than only in isolation. Example workflows include: user login → product search → vendor comparison → add to cart → multi-vendor checkout → order retrieval. Another workflow is product sale → inventory update → sales-pattern analysis → inventory alert.

Hybrid database integration is also important. The backend must correctly retrieve transactional records from MySQL and activity or recommendation records from MongoDB, then combine them without exposing database-specific implementation details to the frontend.

### 8.3 Tools Used for Testing

- Postman for REST API request/response testing.
- Browser developer tools for frontend behaviour and network inspection.
- MySQL Workbench for validating SQL records and constraints.
- MongoDB Compass for inspecting collections and documents.
- Git/GitHub history for tracing changes associated with defects.
- Manual scenario testing for customer and vendor workflows.

### 8.4 Test Cases and Results

| TC ID | Test Case                        | Expected Result                                  | Status |
|-------|----------------------------------|--------------------------------------------------|--------|
| TC01  | Valid user registration          | User record created successfully                 | Pass   |
| TC02  | Invalid login credentials        | Authentication rejected with clear error         | Pass   |
| TC03  | Search product by keyword        | Matching products returned                       | Pass   |
| TC04  | Compare multiple vendors         | Vendor offers displayed with comparison metrics  | Pass   |
| TC05  | Add products from two vendors    | Single cart retains both vendor identities       | Pass   |
| TC06  | Checkout with insufficient stock | Checkout rejected; no inconsistent order created | Pass   |
| TC07  | Successful multi-vendor checkout | Order and vendor-specific details created        | Pass   |

| TC ID | Test Case                | Expected Result                                       | Status |
|-------|--------------------------|-------------------------------------------------------|--------|
| TC08  | Price-history retrieval  | Historical values returned in time order              | Pass   |
| TC09  | Recommendation request   | Relevant recommendation records returned              | Pass   |
| TC10  | Inventory threshold rule | Alert generated when rule condition is met            | Pass   |
| TC11  | Unauthorized admin route | Request denied                                        | Pass   |
| TC12  | Invalid API payload      | Validation error returned without database corruption | Pass   |

The test matrix represents the intended validation scenarios for the academic implementation. Actual execution results should be retained in the team's Postman collection and database logs.

## CHAPTER - 9 DEPLOYMENT

### 9.1 Steps to Deploy

Deployment begins by preparing the source repository and installing the required Node.js dependencies. The MySQL schema is created using SQL scripts or MySQL Workbench, and the MongoDB database is initialized with the required collections. Environment variables are then configured for database connections and authentication secrets.

The Express backend is started in the target environment and tested using Postman. The frontend files are served from the chosen hosting environment and configured to call the deployed API base URL. CORS settings are updated so that the frontend origin is permitted while unknown origins remain restricted.

Before public access, the team should test authentication, product search, vendor matching, cart behaviour, checkout, and order retrieval. Logs and database backups should be enabled before the application is considered ready for wider use.

### 9.2 Environment Configuration

Environment variables should contain values such as the application port, MySQL host, MySQL username, MySQL password, MySQL database name, MongoDB connection string, authentication secret, and any future third-party API credentials. These values must not be hard-coded into source files or committed to GitHub.

Separate development and production configurations reduce the risk of accidentally modifying live data while testing. Database backups should be created before major schema changes.

### 9.3 Hosting of Frontend and Backend

The frontend can be hosted on a static hosting service or served by a web server, while the Express backend can run on a cloud application instance. MySQL and MongoDB can be hosted as managed services or on dedicated servers depending on project scale.

For a production environment, HTTPS should be enabled, database access should be restricted to trusted application servers, and monitoring should be configured for API errors, response time, resource usage, and database availability.

## CHAPTER - 10 CHALLENGES & LIMITATIONS

### 10.1 Issues Faced During Development

One challenge is maintaining consistency when the application uses two database systems. Developers must decide which records belong in MySQL and which belong in MongoDB while avoiding unnecessary duplication. A second challenge is modelling multi-vendor orders because the same customer checkout can contain products with different sellers and fulfilment states.

Another challenge is building meaningful vendor metrics. A Trust Score is only useful if its inputs are measurable and consistently updated. Similarly, price-history analysis requires time-stamped records rather than a single current-price field. Inventory rules must distinguish between normal fluctuations and sustained sales patterns.

API validation and error handling are additional challenges. A backend endpoint must reject malformed input, unauthorized access, and invalid identifiers without exposing internal database details. Testing must also account for cases in which a product becomes unavailable between search and checkout.

### 10.2 Solutions Applied

The hybrid database design separates transaction-critical records from flexible activity data. Clear backend modules and repository/data-access functions reduce the chance that frontend code becomes coupled directly to database implementation.

For multi-vendor checkout, cart lines preserve vendor identity and checkout groups items by vendor before order records are created. Validation is performed again at checkout so that current inventory and price information is authoritative.

For vendor analytics, metrics are normalized before being combined into matching or Trust Score calculations. This avoids directly adding values with different units, such as rupee price, percentage completion, and five-point ratings.

### 10.3 Current Limitations or Known Constraints

The academic prototype depends on the quality and completeness of the data entered into the database. A Trust Score or recommendation result cannot be meaningful if there are too few reliable historical records. Likewise, price-history insights are limited during the early stages of operation.

The current design describes a marketplace architecture but does not by itself guarantee production-scale payment processing, logistics integration, fraud detection, or high-availability infrastructure. These capabilities require additional services, operational monitoring, security review, and real-world testing.

Recommendation quality may initially be rule-based rather than machine-learning based. Inventory alerts may also rely on configurable thresholds until sufficient historical sales data exists for more advanced

forecasting.

## CHAPTER - 11 FUTURE ENHANCEMENTS

### 11.1 Planned Features

- AI/ML-based personalized product recommendations.
- Dynamic vendor matching weights based on customer preferences.
- Advanced demand forecasting for inventory management.
- Automated vendor performance dashboards.
- Real-time order tracking and logistics integration.
- Payment gateway integration with secure webhook processing.
- Coupons, promotions, loyalty points, and subscription options.
- Multilingual interface and multi-currency support.
- Mobile application using the same backend APIs.
- Advanced search with semantic and natural-language queries.

### 11.2 Possible Integrations or Optimizations

The marketplace can integrate with logistics providers so that delivery estimates and tracking events are available inside the order page. Payment providers can be connected through tokenized payment flows and server-side webhooks so that payment status is not trusted solely from browser responses.

Recommendation processing can be optimized by precomputing frequently requested recommendations and caching them. MongoDB can retain flexible event histories, while MySQL remains the authoritative store for transactions. Redis or a similar caching layer can later reduce repeated reads for popular products and vendor information.

The matching algorithm can be personalized by allowing customers to choose preferences such as lowest price, fastest delivery, highest-rated vendor, or balanced value. This would make the marketplace more transparent because the result ordering would reflect an explicit user preference rather than a fixed hidden weighting.

For larger deployments, the backend can be divided into independently scalable services for authentication, catalogue, orders, recommendations, notifications, and analytics. Automated CI/CD, database backups, observability, and security scanning can then be introduced as operational requirements.

## CHAPTER - 12 CONCLUSION

### 12.1 Summary of the Project

The Smart Multi-Vendor E-Commerce Marketplace is a database-driven web application designed to bring multiple sellers and customers into a common shopping environment. The project extends beyond basic product listing by incorporating vendor intelligence, transaction-aware cart management, historical price analysis, recommendations, and inventory rules.

The hybrid database architecture is central to the solution. MySQL provides structured storage for users, vendors, products, inventory, orders, payments, and order details, while MongoDB supports flexible activity, recommendation, notification, and configuration-oriented information. Node.js and Express.js provide the backend layer that coordinates these data sources and exposes REST APIs to the frontend.

The platform demonstrates how database design and backend processing can directly improve the usefulness of an e-commerce system. Instead of treating each seller offer as an isolated listing, the application evaluates vendor information, preserves vendor identity through the cart and checkout process, and uses historical records to support additional insights.

### 12.2 What Was Achieved

- A complete conceptual architecture for a multi-vendor marketplace was defined.
- A hybrid MySQL and MongoDB data strategy was established.
- Smart Multi-Vendor Matching and Vendor Trust Score logic were designed.
- Multi-Vendor Cart and vendor-specific order processing were defined.
- Price History Analysis and Smart Inventory Rules were incorporated.
- A REST-based Node.js and Express.js backend structure was documented.
- Postman-based API testing and integration scenarios were specified.
- Development, version control, deployment, and security practices were documented.

### 12.3 Skills Learned During Development

- Relational database design, normalization, keys, relationships, and transactions.
- NoSQL document modelling and flexible data storage.
- REST API design using Node.js and Express.js.
- Frontend development using HTML, CSS, JavaScript, and Bootstrap.
- Authentication, validation, error handling, and backend security concepts.
- Marketplace-specific algorithms and score normalization.

- API testing with Postman and database verification using MySQL Workbench and MongoDB Compass.
- Git and GitHub based collaboration and version control.
- Technical documentation, system design, and project presentation.

## CHAPTER - 13 REFERENCES

1. MySQL Documentation — SQL, transactions, constraints, indexing, and relational database concepts.
2. MongoDB Documentation — collections, documents, indexing, aggregation, and database operations.
3. Node.js Documentation — runtime concepts, modules, asynchronous programming, and package management.
4. Express.js Documentation — routing, middleware, REST API development, and error handling.
5. Bootstrap Documentation — responsive layout, components, utilities, and form design.
6. Postman Documentation — API collections, environments, testing, and request validation.
7. Git Documentation — version control, branching, merging, and collaboration workflows.
8. GitHub Documentation — repositories, pull requests, issues, and collaborative development.
9. MySQL Workbench Documentation — schema design, SQL development, and database administration.
10. MongoDB Compass Documentation — graphical database inspection and document management.

Note: URLs and access dates can be added by the team if the university requires a web-reference format.

## CHAPTER - 14 APPENDICES

### 14.1 Sample SQL Schema

The following sample illustrates the structure of transaction-critical entities. The final schema can be extended with additional constraints, indexes, audit fields, and payment-specific attributes.

```
CREATE TABLE vendors (  
  vendor_id INT PRIMARY KEY AUTO_INCREMENT,  
  vendor_name VARCHAR(120) NOT NULL,  
  rating DECIMAL(3,2) DEFAULT 0,  
  trust_score DECIMAL(5,2) DEFAULT 0  
);  
  
CREATE TABLE products (  
  product_id INT PRIMARY KEY AUTO_INCREMENT,  
  product_name VARCHAR(180) NOT NULL,  
  category VARCHAR(100),  
  description TEXT  
);  
  
CREATE TABLE inventory (  
  inventory_id INT PRIMARY KEY AUTO_INCREMENT,  
  product_id INT NOT NULL,  
  vendor_id INT NOT NULL,  
  quantity INT NOT NULL DEFAULT 0,  
  price DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (product_id) REFERENCES products(product_id),  
  FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id)  
);
```

### 14.2 Sample Express.js API Structure

A modular Express.js project can separate routes, controllers, services, database access, middleware, and configuration. The following simplified example illustrates the style of an endpoint.

```
const express = require("express");  
const router = express.Router();  
  
router.get("/products/:id/offers", async (req, res) => {  
  try {  
    const offers = await vendorService.getOffers(req.params.id);  
    res.status(200).json({ success: true, data: offers });  
  } catch (error) {  
    res.status(500).json({  
      success: false,  
      message: "Unable to retrieve vendor offers"  
    });  
  }  
});  
  
module.exports = router;
```

### 14.3 Sample MongoDB Activity Document

Flexible activity records can be stored as documents. The exact document structure can evolve as new event types are introduced.

```
{
```

```

    "userId": "U1025",
    "eventType": "PRODUCT_VIEW",
    "productId": "P2048",
    "timestamp": "2026-10-03T08:30:00Z",
    "source": "search",
    "metadata": {
      "category": "electronics",
      "query": "wireless headphones"
    }
  }
}

```

#### 14.4 Installation / Setup Instructions

- Install Node.js and npm on the development machine.
- Install MySQL and create the project database using the supplied SQL schema.
- Install MongoDB locally or configure a MongoDB service and create the required database.
- Clone the project repository from GitHub.
- Open the project in Visual Studio Code.
- Run npm install to install backend dependencies.
- Create an environment configuration file containing database and authentication settings.
- Start the Express server using the configured npm script.
- Open the frontend in the configured development server or browser.
- Use Postman to verify authentication, product, vendor, cart, order, and analytics endpoints.

#### 14.5 User Manual / Guide

1. Open the marketplace and register or log in. 2. Search for a product or browse a category. 3. Review the available vendor offers and compare price, availability, rating, delivery performance, return rate, and Trust Score. 4. Add selected offers to the cart. 5. Review the vendor grouping shown in the cart. 6. Proceed to checkout and confirm the order. 7. Open the order history page to review order and vendor-specific details. 8. Use the price-history and recommendation sections where available. 9. Administrators can review vendor, product, inventory, and order information according to their authorized role.

#### 14.6 UI / Evidence Screenshot Slots

The original university template includes application screenshots in the appendix. Because actual application screenshots were not supplied with the project brief, the following slots are intentionally provided for the team to insert real screenshots before submission.

| Figure | Recommended evidence                 |
|--------|--------------------------------------|
| A1     | Home / marketplace landing page      |
| A2     | Product search and vendor comparison |
| A3     | Vendor Trust Score display           |
| A4     | Multi-Vendor Cart                    |

| Figure | Recommended evidence                        |
|--------|---------------------------------------------|
| A5     | Checkout and vendor-specific order grouping |
| A6     | Price History Analysis                      |
| A7     | Product Recommendations                     |
| A8     | Smart Inventory Rules / admin alert         |
| A9     | MySQL Workbench schema                      |
| A10    | MongoDB Compass collections                 |
| A11    | Postman API test result                     |
| A12    | GitHub repository / version history         |

## 14.7 Review Form

Project Title: Smart Multi-Vendor E-Commerce Marketplace

Team Members: G. Neha Deepika (2520030068), M. Thara Varshini (2520030052)

Course: Database Systems Engineering and Distributed Backend Development (25CS1302E)

Course Instructor: Mr. Sateesh Kumar

| Review Item            | Comments / Suggestions | Signature |
|------------------------|------------------------|-----------|
| Project definition     |                        |           |
| Database design        |                        |           |
| Backend implementation |                        |           |
| Testing and validation |                        |           |
| Documentation quality  |                        |           |
| Overall review         |                        |           |

Guide / Course Instructor Signature: \_\_\_\_\_

Date: \_\_\_\_\_ Place: \_\_\_\_\_